

2-5 · Python 기반의 크롤러 · 실습 1H

requests·BeautifulSoup 기초 실습

robots.txt 확인부터 한 페이지 파싱까지

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1시간 뒤에는

- ① 가상환경에 requests·bs4·pymysql 을 설치하고 ok 를 확인한다
- ② robots.txt 확인을 코드의 첫 단계로 넣는다
- ③ requests 로 받고 BeautifulSoup 으로 명언 10건을 뽑는다
- ④ 없는 선택자의 NoneType 오류를 겪고 방어 코드를 쓴다

다음 페이지 링크(li.next a)의 None 이 다음 1.5H 파이프라인의 종료 조건이 됩니다

여섯 단계 — 받는 것과 해석하는 것을 나눠서

실습점검을 마친 (.venv) 터미널에서 — 파일은 전부 ~/data-eng/week2 에 저장합니다

1. 환경 준비

pip install
docker start

2. robots.txt

check_robots.py

3·4. 받기·뽑기

fetch_raw.py
fetch_one.py

5. 일부러 실패

없는 선택자
→ NoneType

6. 다음 페이지

li.next a
None 이면 끝

1단계

핵심 명령 — 설치 · 확인 · MySQL 기동

가상환경	source .venv/bin/activate — OS 셋 같다
설치	pip install requests beautifulsoup4 pymysql (점검에서 했으면 건너뛰)
확인	import requests, bs4, pymysql → ok
MySQL	docker start de-mysql
파일 저장	code 파일.py 또는 nano 파일.py → python3 파일.py

확인 포인트: ok 가 출력되고 de-mysql 이 기동되면 준비 완료

수집 가능 여부: True — 파일이 없어서 나온 값

1 출력: 수집 가능 여부: True · 권고 간격: None
RobotFileParser 가 robots.txt 를 읽어 can_fetch 로 판정한다

2 그런데 브라우저로 열면 404
파일이 없으면 「막는 규칙이 없다」로 보고 전부 허용 — 401·403 이면 반대로 전부 금지

3 권고 간격이 None 이어도 1초를 둔다
파일이 없는 것과 허용한다고 적힌 것은 다르다 — 다른 사이트는 약관까지 확인

상태 코드에 따라

**결론이
뒤집힌다**

fetch_raw.py → fetch_one.py — 받기와 해석을 나눈다

줄	뜻
headers = {'User-Agent': ...}	정체와 목적을 밝힌다
requests.get(..., timeout=10)	받기만 — 파싱은 아직
res.status_code	200 이면 성공
BeautifulSoup(res.text, ...)	문자열을 태그 트리로
soup.select('div.quote')	명언 덩어리 10개
q.select_one('span.text')	본문 하나 — get_text()
[... for t in q.select('a.tag')]	태그 목록을 한 줄로

[... for t in ...] 리스트 컴프리헨션 = 빈 목록에 for 로 append 한 것과 같은 결과

확인 포인트: status: 200 뒤에 「저자 | 본문 앞부분 | 태그 목록」 10건이 찍힌다

없는 선택자를 써 본다 — NoneType 오류를 미리 겪는다

selector_test.py 는 터지는 것이 목적 — 방어 코드는 safe_pick.py 로 따로, 6단계는 next_page.py

select 오타	select_one 오타	.get_text()	safe_pick.py
'div.quotes' 없음 → 빈 리스트 []	'span.txt' 없음 → None	None 에 호출 → AttributeError	새 파일 — found 검사 후 (없음)

터지는 코드와 방어하는 코드

터지는 쪽 — `.get_text()` 바로 호출

None 에 `.get_text()` 를 부르는 순간 터진다

- `select` → 빈 리스트 []
- `select_one` → None
- `AttributeError: 'NoneType' object ...`

VS

방어하는 쪽 — `pick()` 헬퍼

없을 수 있는 값은 검사하고 넘어간다

- `found = el.select_one(selector)`
- 있으면 `get_text(strip=True)`
- 없으면 default — (없음), 멈추지 않음

li.next a — 다음 페이지 링크, None 이면 끝

- 1 첫 페이지: next href 가 /page/2/**
soup.select_one('li.next a') 의 href 를 읽는다
- 2 마지막 페이지(/page/10/): None**
select_one 이 None 을 돌려준다 — 이것이 순회를 멈추는 신호
- 3 다음 1.5H 에서 반복 종료 조건으로 쓴다**
next_url 이 없으면 break — 있으면 그 주소로 계속

확인 포인트

**/page/2/
그리고 None**

이 1시간이 끝날 때 확인할 것

- 1 import 확인이 ok 를 찍고 de-mysql 이 떠 있다
- 2 check_robots.py 가 True · None 을 찍는다
- 3 fetch_raw.py 가 status: 200 과 HTML 앞부분을 보여 준다
- 4 fetch_one.py 가 명언 10건을 저자 | 본문 | 태그로 찍는다
- 5 없는 선택자는 (없음)으로 넘어가고 li.next a 는 None 을 만난다

2-5 · 실습 1H 마무리

다음은 수집 파이프라인 구축 (1.5H)

fetch·parse·load 세 함수로 나눈 크롤러를 완성하고, 재실행해도 중복이 안 쌓이는지 확인합니다

✓ quotes 표 + UNIQUE 제약 만들기

✓ crawler_pipeline.py 로 3페이지 수집·적재

✓ 한 번 더 실행해 saved 0 확인

✓ 재시도·백오프 안전장치 붙이기

가져갈 것

fetch_one.py 의 선택자와 헤더